VISUME

Video Resume Job Platform

HRTech | Problem Statement #03
GradSkills Summership Challenge 2026

Complete Project Blueprint & Build Guide
May 24–26, 2026 | 48-Hour Sprint Edition

1. Platform Overview

Visume is a video-first hiring platform that replaces flat PDF resumes with recorded video profiles. Candidates showcase their personality and communication skills through video. Recruiters get AI-matched ranked lists of candidates, a Kanban hiring pipeline, and virtual interview scheduling — all in one unified system.

1.1 The Problem We Solve

Problem	Impact	Our Solution
PDF resumes hide personality	Poor hiring decisions	Video resume recording
Recruiters waste time screening	High cost-per-hire	AI match scoring
Disconnected hiring tools	Missed candidates	Unified recruiter dashboard
Candidates can't stand out	Low response rates	Rich media profiles
No structured pipeline	Disorganised hiring	Kanban pipeline board

1.2 Why This Wins the Competition

- Fewest teams selected (6/40) — lowest competition among all 6 problems
- Video + AI combination creates an unforgettable 5-minute live demo
- Directly aligned with real market demand — PDF resumes are genuinely broken
- Dual-portal architecture demonstrates full-stack system design thinking
- Core user journey can be demonstrated end-to-end in under 3 minutes

2. System Architecture

2.1 Architecture Layers

Layer	Technology	Purpose
Frontend	Next.js 15 + Tailwind CSS + Shadcn UI	Candidate Portal, Recruiter Portal, Landing Page
API Layer	Next.js API Routes	Auth, Profile, Jobs, AI Match, Video endpoints
Database	PostgreSQL via Supabase + Prisma ORM	All persistent data storage
Cache	Redis (Upstash free tier)	Session data, rate limiting
Video Storage	Cloudinary	Video resume upload, streaming, thumbnails
AI Matching	OpenAI Embeddings API	Skill vector matching, candidate scoring
Authentication	NextAuth.js	Role-based sessions (Candidate / Recruiter)
KYC (Mocked)	Custom upload flow + badge	Identity verification UI
Deployment	Vercel	CI/CD, edge functions, environment management
Notifications	Nodemailer / Resend	Email alerts for applications and interviews

2.2 Data Flow — Core Journey

The complete user journey that will be demonstrated to judges:

1. Candidate registers and completes mocked KYC verification
2. Candidate records or uploads their video resume (stored on Cloudinary)
3. Candidate fills profile — skills, experience, headline, location
4. AI service generates embedding vector from candidate skills
5. Recruiter posts a job with required skills
6. AI compares job embedding vs all candidate embeddings
7. Recruiter sees candidates ranked by AI match percentage
8. Recruiter shortlists candidates and moves them through Kanban board
9. Interview is scheduled and candidate receives email notification

3. Database Schema

3.1 Core Tables

```
model User {
  id         String    @id @default(cuid())
  email      String    @unique
  name       String
  phone      String?
  role       Role      // CANDIDATE | RECRUITER | ADMIN
  kycStatus  KYCStatus // PENDING | VERIFIED | REJECTED
  createdAt  DateTime  @default(now())
}

model CandidateProfile {
  id           String   @id @default(cuid())
  userId       String   @unique
  headline     String?
  location     String?
  bio          String?
  skills       String[]
  videoResume  String?  // Cloudinary URL
  pdfResume    String?  // Cloudinary URL
  embedding    Float[]  // AI vector from skills
  applications Application[]
}

model RecruiterProfile {
  id          String @id @default(cuid())
  userId      String @unique
  companyName String
  companyLogo String?
  industry    String?
  jobs        Job[]
}

model Job {
  id          String   @id @default(cuid())
  recruiterId String
  title       String
  description String
  skills      String[]
  location    String
  salaryMin   Int?
  salaryMax   Int?
  workMode    WorkMode // REMOTE | HYBRID | ONSITE
  isActive    Boolean  @default(true)
  embedding   Float[]  // AI vector from required skills
  applications Application[]
  createdAt   DateTime @default(now())
}

model Application {
  id          String    @id @default(cuid())
  candidateId String
  jobId       String
  status      AppStatus // APPLIED | REVIEWED | SHORTLISTED | REJECTED | HIRED
  aiScore     Float?    // 0-100 match percentage
  createdAt   DateTime  @default(now())
```

```
  interview   Interview?
}

model Interview {
  id            String          @id @default(cuid())
  applicationId String          @unique
  scheduledAt   DateTime
  meetLink      String?
  notes         String?
  status        InterviewStatus // SCHEDULED | COMPLETED | CANCELLED
}
```

4. Project Structure

```
video-resume-platform/
├── app/
│   ├── (auth)/
│   │   ├── login/              ← Login page
│   │   ├── register/           ← Register with role selector
│   │   └── kyc/                ← KYC upload flow (mocked)
│   ├── candidate/
│   │   ├── dashboard/          ← Stats + recommended jobs
│   │   ├── profile/            ← Edit profile + skills
│   │   ├── video-resume/       ← Record or upload video
│   │   ├── jobs/               ← Browse + filter jobs
│   │   └── applications/       ← Track application status
│   ├── recruiter/
│   │   ├── dashboard/          ← Stats + AI-matched candidates
│   │   ├── candidates/         ← Search + view profiles
│   │   ├── pipeline/           ← Kanban board
│   │   ├── jobs/               ← Post + manage jobs
│   │   └── company/            ← Company profile
│   └── api/
│       ├── auth/               ← NextAuth endpoints
│       ├── profile/            ← CRUD for profiles
│       ├── jobs/               ← Job CRUD + application
│       ├── match/              ← AI matching logic
│       └── video/              ← Cloudinary upload handler
├── components/
│   ├── ui/                     ← Shadcn base components
│   ├── candidate/              ← Candidate-specific components
│   ├── recruiter/              ← Recruiter-specific components
│   └── shared/                 ← Shared across portals
├── lib/
│   ├── db.ts                   ← Prisma client singleton
│   ├── ai.ts                   ← OpenAI embedding helpers
│   ├── cloudinary.ts           ← Video upload helpers
│   └── auth.ts                 ← NextAuth config + role guards
├── prisma/
│   └── schema.prisma
└── public/
```

5. Feature Breakdown

5.1 Candidate Portal Features

Feature	Description	Priority
Registration + Role Select	Email/password signup with CANDIDATE role selection	P1 — Must Have
KYC Verification (Mocked)	Upload Aadhaar/PAN UI, verified badge stored in DB	P1 — Must Have
Video Resume Recorder	MediaRecorder API in-browser recording + Cloudinary upload	P1 — Must Have
Profile Builder	Headline, bio, skills multi-tag, location, photo upload	P1 — Must Have
Job Discovery	Browse jobs with filter by skills, location, work mode	P1 — Must Have
AI Match Score	See % match score on each job listing	P1 — Must Have
One-Click Apply	Apply to job, status tracked in Applications tab	P1 — Must Have
Application Tracker	See status: Applied, Reviewed, Shortlisted, Interview	P2 — Should Have
PDF Resume Upload	Alternative for candidates without video	P3 — Nice to Have

5.2 Recruiter Portal Features

Feature	Description	Priority
Registration + Company Setup	Recruiter signup with company name, logo, industry	P1 — Must Have
Post a Job	Title, description, required skills, salary, work mode	P1 — Must Have
AI-Matched Candidate List	Ranked list of candidates sorted by match % per job	P1 — Must Have
Candidate Video Player	Watch candidate video resume directly in dashboard	P1 — Must Have
Kanban Pipeline	Drag candidates through Applied, Reviewed, Shortlisted, Interview, Hired	P1 — Must Have
Shortlist + Reject	Quick action buttons on candidate cards	P1 — Must Have
Schedule Interview	Set date/time, generates mock meet link, notifies candidate	P2 — Should Have

Feature	Description	Priority
Candidate Search	Search by name, skill, or location across all applicants	P2 — Should Have

6. AI Matching Engine

6.1 How It Works

The AI matching engine uses OpenAI's text-embedding-ada-002 model to convert skill lists into high-dimensional vectors. Cosine similarity between job and candidate vectors produces a match percentage shown to recruiters.

6.2 Implementation

```
// lib/ai.ts
import OpenAI from 'openai';
const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });

// Generate embedding from skills array
export async function generateEmbedding(skills: string[]): Promise<number[]> {
  const text = skills.join(', ');
  const response = await openai.embeddings.create({
    model: 'text-embedding-ada-002',
    input: text,
  });
  return response.data[0].embedding;
}

// Calculate cosine similarity between two vectors
export function cosineSimilarity(a: number[], b: number[]): number {
  const dot = a.reduce((sum, val, i) => sum + val * b[i], 0);
  const magA = Math.sqrt(a.reduce((sum, val) => sum + val * val, 0));
  const magB = Math.sqrt(b.reduce((sum, val) => sum + val * val, 0));
  return dot / (magA * magB);
}

// Get AI match score as percentage (0-100)
export function matchScore(jobEmbedding: number[], candidateEmbedding: number[]):
number {
  const similarity = cosineSimilarity(jobEmbedding, candidateEmbedding);
  return Math.round(((similarity + 1) / 2) * 100);
}
```

6.3 When Embeddings Are Generated

- Candidate embedding: generated when candidate saves their profile skills
- Job embedding: generated when recruiter posts a new job
- Match score: computed on-the-fly when recruiter views candidates for a specific job
- Scores are cached in the Application table once computed to avoid repeat API calls

7. Complete Tech Stack

Category	Technology	Version	Why
Frontend Framework	Next.js	15 (App Router)	SSR, API routes, file-based routing
Styling	Tailwind CSS	3.x	Utility-first, rapid dark theme
UI Components	Shadcn UI	Latest	Accessible, customisable base
Database ORM	Prisma	5.x	Type-safe DB queries
Database Host	Supabase (PostgreSQL)	Free tier	Instant setup, no DevOps
Authentication	NextAuth.js	5.x (beta)	Role-based sessions
Video Storage	Cloudinary	Free tier	Upload, transform, stream video
AI Matching	OpenAI API	ada-002 model	Text embeddings for skill matching
Email	Resend	Free tier	Interview notification emails
Deployment	Vercel	Free tier	Zero-config Next.js deployment
Icons	Lucide React	Latest	Consistent icon system
Drag and Drop	dnd-kit	Latest	Kanban board drag interactions
Form Handling	React Hook Form + Zod	Latest	Validated forms

8. 48-Hour Build Plan

COMPETITION DEADLINE
Final submission: May 26, 2026. You have exactly 48 hours from May 24. Build the core demo journey first — every hour counts.

8.1 What to Build (Core Demo Journey)

Judges spend 5-8 minutes per project. Build this complete end-to-end story and build it well:

10. Candidate registers → completes KYC → records video resume
11. Recruiter posts a job → AI ranks candidates → views ranked list
12. Recruiter moves candidate through Kanban → schedules interview

8.2 What to Cut (Judges Won't See It)

Cut Feature	Reason	Mock Instead
Real Digilocker KYC API	Complex OAuth + paid API	Upload UI + hardcoded 'Verified' badge
Google Calendar integration	OAuth setup takes hours	Date picker + mock meet link generator
Redis caching	Not visible to judges	Skip entirely, use DB directly
Admin dashboard	Third portal = triple the work	Skip entirely
PDF resume AI parsing	Complex pipeline	Simple file upload only
Real-time chat	WebSocket complexity	Skip entirely
Payment/billing	Not relevant to demo	Skip entirely

8.3 Hour-by-Hour Schedule

DAY 1 — May 24 (Today)

Hours	Task	Output
Hour 1-2	Project setup: Next.js 15 + Tailwind + Shadcn + Prisma + Supabase	Running dev server with DB connected
Hour 3-4	NextAuth setup with CANDIDATE and RECRUITER roles. Login + Register pages (use Stitch prompt from Section 9)	Auth working, role-based redirect
Hour 5-6	KYC mock page + Candidate dashboard layout + sidebar (Stitch)	Candidate can log in and see dashboard

Hours	Task	Output
Hour 7-8	Video resume recorder: MediaRecorder API + Cloudinary upload + save URL to DB	Candidate can record and save video
Hour 9-10	Candidate profile page: skills, bio, headline, photo (Stitch + code). AI embedding generated on save	Full candidate profile working
Hour 11-12	Job listing page + job cards + one-click apply (Stitch + code)	Candidate can browse and apply to jobs

DAY 2 — May 25

Hours	Task	Output
Hour 1-2	Recruiter registration + company setup + dashboard layout (Stitch)	Recruiter portal skeleton done
Hour 3-4	Job posting form: title, skills, description, work mode (Stitch + code). Generate job embedding on save.	Recruiter can post jobs
Hour 5-6	AI matching API: compute cosine similarity, rank candidates. Recruiter sees ranked list with % badges	Core AI feature working
Hour 7-8	Candidate video player in recruiter view. Shortlist / Reject quick actions.	Recruiter can watch video resumes
Hour 9-10	Kanban pipeline board with dnd-kit. Drag candidates between stages.	Full Kanban working
Hour 11-12	Interview scheduling: date picker + mock meet link + email via Resend. Application status updates.	End-to-end journey complete

DAY 3 Morning — May 26 (Polish)

Hours	Task	Output
Hour 1-2	Landing page (Stitch). Fix any broken flows. Seed realistic demo data.	Platform looks complete
Hour 3-4	Deploy to Vercel. Set all environment variables. Test full journey on production URL.	Live demo URL ready
Hour 5+	Prepare demo script. Record walkthrough video as backup.	Ready to present

9. Stitch UI Guide — Complete Design System

9.1 Master Design Tokens

Paste these tokens at the start of EVERY Stitch prompt to ensure consistency across all pages:

```
Design System Rules — apply to ALL pages without exception:

Font: 'DM Sans' for all body text and UI
      'Syne' for all page headings and names only
Background: #0A0A0F (near-black, entire page bg)
Card background: #13131A
Border color: #1E1E2E
Primary accent: #6C5CE7 (electric purple)
Secondary accent: #00CEC9 (teal)
Success color: #00B894
Danger color: #D63031
Text primary: #EAEAF0
Text muted: #6B6B80
Border radius: 12px for all cards, 8px for all inputs, 999px for badges/pills
Spacing unit: 8px grid (use 8, 16, 24, 32, 48, 64px only)
Button primary: #6C5CE7 bg, white text, 8px radius, 12px 24px padding
Button outline: transparent bg, #6C5CE7 border and text, same padding
All inputs: bg #1E1E2E, border #2A2A3E, text #EAEAF0, focus border #6C5CE7
Sidebar width: 240px, fixed left, bg #13131A
Active sidebar item: left purple border 3px + bg rgba(108,92,231,0.12)
Icon set: Lucide icons ONLY — no other icon libraries
Dark theme ONLY — no white backgrounds, no light cards, no light inputs
No lorem ipsum — use realistic placeholder content
```

9.2 Page-by-Page Stitch Prompts

Prompt 1 — Landing Page

```
Build a dark-themed landing page for 'Visume' — a video resume job platform.
Use DM Sans body font, Syne for headings.
Background #0A0A0F, accent #6C5CE7, teal #00CEC9.

Sections:
Navbar: Logo 'Visume' left, nav links center (How it Works, For Recruiters,
Pricing),
  'Post a Job' outlined button + 'Record Resume' purple filled button right.
  Glassmorphism navbar with blur and subtle border.

Hero: Large Syne heading 'Get Hired by Being Seen', subtext about video resumes.
  Two CTA buttons: purple filled 'Record My Resume' + teal outline 'Browse Jobs'.
  Right side: floating mock video resume card with candidate photo, play button,
  name, headline, and AI match badge '94% Match' in teal.

Features: 3-column grid with dark icon cards:
  - Video Resume: Record yourself in 60 seconds
  - AI Matching: Smart skill-based scoring
  - Instant Interviews: Recruiters reach you directly

Stats bar: dark bg, three large numbers:
```

```
  2,400+ Candidates | 380+ Companies | 94% Match Accuracy

Footer: minimal dark, logo left, links right.

[Apply global design tokens from Section 9.1]
```

Prompt 2 — Candidate Registration + KYC

```
Build a multi-step registration flow for job candidates. Dark theme only.
bg #0A0A0F, card bg #13131A, accent #6C5CE7. Font: DM Sans.

Show 3-step progress bar at top with step labels:
  Step 1: Account Setup | Step 2: KYC Verification | Step 3: Profile

Step 1 — Account Setup:
  Full Name, Email Address, Password, Confirm Password, Phone Number fields.
  Role selector: two cards side by side — 'I'm a Candidate' and 'I'm a Recruiter'.
  Active role card has purple border. Purple 'Continue' button.

Step 2 — KYC Verification:
  Heading 'Verify Your Identity'. Two upload zones side by side:
    Left: 'Upload Aadhaar Card' with upload icon
    Right: 'Upload PAN Card' with upload icon
  Selfie section below: camera icon button 'Take Selfie'.
  Status badge below each upload: 'Pending Verification' in yellow.
  'Submit for Verification' purple button.

Step 3 — Profile Setup:
  Profile photo upload circle with edit icon overlay.
  Professional Headline input, Skills multi-select tag input,
  Location dropdown, LinkedIn URL input.
  'Complete Profile' purple button.

[Apply global design tokens from Section 9.1]
```

Prompt 3 — Candidate Dashboard

```
Build a candidate dashboard with fixed left sidebar. Dark theme.
Page bg #0A0A0F, sidebar bg #13131A, cards #13131A. Font: DM Sans.

Sidebar (240px fixed):
  Top: Visume logo.
  Nav items with Lucide icons:
    Dashboard (active), My Profile, Video Resume, Browse Jobs, Applications,
Settings.
  Active: 3px purple left border + rgba(108,92,231,0.12) bg.
  Bottom: user avatar + name + 'KYC Verified' green badge + logout.

Main content area:
  Header: 'Welcome back, Ashwin' (Syne font) + small 'KYC Verified' badge.

  Stats row (4 cards, equal width):
    Jobs Applied: 12 | Profile Views: 284 | Interview Calls: 3 | Top Match: 97%
    Each card: large number in accent color, label below, subtle icon top-right.
```

```
  Video Resume section: dark card with 16:9 video thumbnail left,
    right side shows title, upload date, view count, 're-record' and 'Replace'
buttons.
    If no video: dashed border card with camera icon and 'Record Now' purple
button.

  Recommended Jobs (grid, 2 columns, 3 jobs):
    Each card: company logo circle, job title (Syne), company name, location tag,
    work mode tag (Remote/Hybrid), salary range, AI match % badge in teal,
    top 3 required skills as small pills, 'Apply Now' button.
    Hover: subtle purple glow border.

[Apply global design tokens from Section 9.1]
```

Prompt 4 — Video Resume Recorder

```
Build a video resume recording page. Dark theme only.
bg #0A0A0F, panel bg #13131A. Accent #6C5CE7. Font: DM Sans + Syne headings.

Two-column layout:

Left panel (60% width):
  16:9 rounded card showing camera feed area (dark placeholder with camera icon).
  When recording: red REC dot top-left with pulsing animation.
  Timer display below feed: 00:00 / 02:00 in large monospace font.
  Controls row: circular buttons —
    Red circle 'Record', grey 'Stop', ghost 'Re-record'.
  Teleprompter toggle below: 'Show Script Tips' switch.
  When toggled: shows overlay card at bottom of feed with tip text.

Right panel (40% width):
  Card: 'Tips for a Great Video Resume' with 5 checklist items:
    - Keep it under 90 seconds
    - Good lighting, quiet background
    - State your name and top skill
    - Smile and speak clearly
    - End with a strong call to action
  Divider with 'OR' text.
  Upload zone: dashed border card 'Upload Existing Video' with upload icon.
  'Save & Publish Resume' purple full-width button at bottom.

[Apply global design tokens from Section 9.1]
```

Prompt 5 — Job Discovery Page

```
Build a job discovery search page for candidates. Dark theme.
bg #0A0A0F, sidebar #13131A, cards #13131A. Accent #6C5CE7. Font: DM Sans.

Left sidebar (280px, filter panel):
  Search input at top with search icon.
  Filter sections with headers:
    Role Type: Remote / Hybrid / Onsite toggle chips.
    Salary Range: dual-handle range slider, shows min-max values.
    Experience Level: checkboxes — Fresher, 1-3 yrs, 3-5 yrs, 5+ yrs.
```

```
    AI Match Filter: toggle switch 'Show only 80%+ matches' (teal when on).
  'Apply Filters' purple button at bottom.

Main area:
  Top bar: 'Showing 24 jobs' + sort dropdown.
  Active filter pills row below (e.g. 'Remote x', '80%+ match x').
  2-column job card grid:
    Each card: company logo, company name muted, job title in Syne font,
    location + work mode tags, salary range, posted date,
    AI match % in large teal with progress bar,
    top 4 required skills as pills,
    'View Details' ghost button + 'Quick Apply' purple button.
    Purple glow on hover.

[Apply global design tokens from Section 9.1]
```

Prompt 6 — Recruiter Dashboard

```
Build a recruiter dashboard with fixed left sidebar. Dark theme.
bg #0A0A0F, sidebar #13131A, cards #13131A. Accent #6C5CE7. Font: DM Sans.

Sidebar (240px fixed):
  Visume logo + 'Recruiter' role badge.
  Nav items: Dashboard (active), Find Candidates, Job Postings,
    Pipeline, Company Profile, Settings.
  Bottom: company logo + name.

Main area:
  Header: 'Good morning, Sarah' (Syne) + 'Post New Job' purple button right.

  Stats row (4 cards):
    Active Jobs: 8 | Total Applicants: 147 | Interviews Today: 4 | Hires This
Month: 2

  AI-Matched Candidates section:
    Heading + 'For: Senior React Developer' job selector dropdown.
    Horizontal scroll row of candidate cards:
      Each card: profile photo (circular), name (Syne), headline,
      top 3 skill pills, large teal '94% Match', star rating,
      'View Profile' ghost + 'Shortlist' purple buttons.

  Recent Activity feed (right side, 30% width):
    Timeline list: avatar + action text + time ago.
    Examples: 'John applied for React Dev', 'AI shortlisted 3 candidates',
    'Interview scheduled with Priya'.

[Apply global design tokens from Section 9.1]
```

Prompt 7 — Hiring Pipeline (Kanban)

```
Build a Kanban hiring pipeline board for recruiters. Dark theme.
bg #0A0A0F, column bg #13131A, cards #1A1A24. Font: DM Sans.

Full-width horizontal board, 5 columns with horizontal scroll:
```

```
Column headers (sticky, with count badge):
  Applied (blue badge) | Under Review (yellow) | Shortlisted (purple)
  Interview Scheduled (teal) | Hired (green)

Each candidate card (draggable):
  Profile photo (circular, 36px) + name + job title applied for.
  AI match % badge top-right.
  Application date muted at bottom.
  Quick action icons row: Eye (view profile), Calendar (schedule interview),
    Check (move next stage), X (reject).
  Cards slightly lighter bg than column.

Add 3-4 sample candidate cards in each column for realism.
Column width: 260px minimum with gap between.
Column header: bold label + count chip + 'Add' icon.
Drag handle visible on hover (grip dots icon, left side).

[Apply global design tokens from Section 9.1]
```

Prompt 8 — Candidate Public Profile

```
Build a public candidate profile view page. Dark theme.
bg #0A0A0F, cards #13131A. Accent #6C5CE7, teal #00CEC9. Font: DM Sans + Syne.

Top hero card:
  Left: profile photo (large, circular, 96px), online status dot.
  Centre: name in Syne 32px, headline below, location with pin icon,
    KYC Verified badge in green, LinkedIn icon link.
  Right: 'Download PDF Resume' ghost button + 'Schedule Interview' purple button.
  Background: subtle purple gradient overlay on dark card.

Video Resume section:
  Full-width 16:9 dark video player card with custom dark controls.
  Title 'Video Resume' above, duration and view count below.

Two-column layout below:
  Left (60%): Skills section (tag pills in #1E1E2E with purple text),
    Experience timeline cards, Education timeline cards.
  Right (40%): Portfolio cards with project name, description, tech tags,
    live link button. Stats card: Applications sent, Match rate, Response rate.

All cards #13131A, borders #1E1E2E, 12px radius.

[Apply global design tokens from Section 9.1]
```

10. Environment Variables

Create a .env.local file in the root of your project with these variables:

```
# Database
DATABASE_URL=postgresql://[user]:[password]@[host]/[db]?pgbouncer=true
DIRECT_URL=postgresql://[user]:[password]@[host]/[db]

# NextAuth
NEXTAUTH_SECRET=your-32-char-random-secret
NEXTAUTH_URL=http://localhost:3000

# Cloudinary (video storage)
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret

# OpenAI (AI matching)
OPENAI_API_KEY=sk-your-openai-key

# Resend (email notifications)
RESEND_API_KEY=re_your-resend-key
FROM_EMAIL=noreply@visume.com
```

10.1 Where to Get Each Key

Service	Where to Get	Free Tier
Supabase (PostgreSQL)	supabase.com → New Project → Settings → Database	500MB, plenty for demo
Cloudinary	cloudinary.com → Sign up → Dashboard → API Keys	25GB storage free
OpenAI	platform.openai.com → API Keys	Paid, ~$0.002 per 1K tokens
Resend	resend.com → API Keys	100 emails/day free
Vercel	vercel.com → Project Settings → Environment Variables	Free hobby tier

11. Demo Script for Judges

11.1 The 5-Minute Story

Practice this exact flow. Every step should work live with no hesitation.

Time	Action	What Judges See
0:00-0:30	Open landing page, explain the problem in 2 sentences	Polished landing, clear value proposition
0:30-1:30	Log in as Candidate → show profile → play video resume	'Wow, actual video playing — this is real'
1:30-2:00	Show Jobs page with AI match scores on each card	Teal percentages — AI is working visibly
2:00-2:30	Click Apply on a high-match job — status changes instantly	End-to-end flow is seamless
2:30-3:30	Switch to Recruiter portal → show AI-ranked candidates list	Highest match at top — AI ranking visible
3:30-4:30	Open Kanban board → drag candidate to 'Interview Scheduled'	Satisfying drag interaction, pipeline works
4:30-5:00	Show interview scheduled email arriving in demo inbox	Complete journey — from apply to interview

PRO TIP
Seed your database with 5-6 realistic candidate profiles and 3-4 jobs BEFORE the demo. Never demo with empty state. Have a backup screen recording ready in case of network issues.

11.2 Likely Judge Questions + Answers

Question	Answer
How does the AI matching work?	OpenAI embeddings convert skill lists into vectors. Cosine similarity gives a 0-100% match score between job requirements and candidate skills.
Is the KYC real?	The UI flow is real — upload, selfie, verification status. For the competition we use a mocked verified state to keep scope focused. In production we'd integrate Digilocker.
How does video storage work?	Videos are recorded via browser MediaRecorder API and uploaded directly to Cloudinary. We store only the Cloudinary URL in our DB, not the raw video.
What makes this different from LinkedIn?	LinkedIn shows text profiles. Visume shows you the actual person speaking — personality, confidence, communication — in 90 seconds. Plus AI pre-ranking saves recruiters hours of screening.

Question	Answer
Can it scale?	Yes — Supabase PostgreSQL scales automatically. Cloudinary handles global video CDN. Vercel edge functions scale to millions of requests.

12. All Abbreviations Expanded

Abbreviation	Full Form
API	Application Programming Interface
AI	Artificial Intelligence
ML	Machine Learning
CRM	Customer Relationship Management
ERP	Enterprise Resource Planning
SaaS	Software as a Service
CMS	Content Management System
RBAC	Role-Based Access Control
LMS	Learning Management System
PDF	Portable Document Format
KYC	Know Your Customer
SVG	Scalable Vector Graphics
ORM	Object-Relational Mapper
SSR	Server-Side Rendering
CDN	Content Delivery Network
DB	Database
URL	Uniform Resource Locator
UI	User Interface
UX	User Experience
CI/CD	Continuous Integration / Continuous Deployment
JWT	JSON Web Token
OAuth	Open Authorization
DXA	Device-independent pixel eXtended Approximation (Word measurement unit)
SMB	Small and Medium-sized Business
POS	Point of Sale
ROI	Return on Investment
QR	Quick Response (code)
HR	Human Resources
MCP	Model Context Protocol
RAG	Retrieval-Augmented Generation

Abbreviation	Full Form
LLM	Large Language Model
ada-002	OpenAI's text-embedding-ada-002 model (embedding generation)
DXA	Device-independent pixel eXtended Approximation
EMU	English Metric Units (Word image sizing)
TOC	Table of Contents

13. Quick Reference — Links & Resources

13.1 Setup Links

- Next.js 15 docs: nextjs.org/docs
- Supabase dashboard: app.supabase.com
- Cloudinary console: console.cloudinary.com
- OpenAI API keys: platform.openai.com/api-keys
- Resend dashboard: resend.com/emails
- Vercel deployment: vercel.com/new
- Shadcn UI: ui.shadcn.com
- dnd-kit docs: dndkit.com

13.2 NPM Packages to Install

```
npm install next@latest react@latest react-dom@latest
npm install @prisma/client prisma
npm install next-auth@beta
npm install @dnd-kit/core @dnd-kit/sortable @dnd-kit/utilities
npm install openai
npm install cloudinary
npm install resend
npm install react-hook-form zod @hookform/resolvers
npm install lucide-react
npx shadcn@latest init
```

FINAL REMINDER
Build the demo journey first. Get video recording working. Get AI scores showing. Get Kanban dragging. Everything else is decoration. Ship early, polish later. Good luck — go win this.